المدرسة الوطنية العليا
للتكنولوجيات المتقدمة

Ecole Nationale Supérieure
des Technologies Avancées

1. Executive Introduction to Geometric Optics in Rendering

In the synthesis of photorealistic imagery, the **Geometric Optics** model serves as the primary mathematical framework for simulating light transport. This model abstracts the complex wave-particle duality of light into a simplified flow of discrete photons propagating along *light rays*, a representation sufficient for the wavelength scales encountered in visible computer graphics.

The model rests on four fundamental postulates:

- **Rectilinear Propagation:** Light rays travel in straight lines within any vacuum or homogeneous medium.
- **Non-Interference:** Crossing rays do not interact or modify each other's trajectories.
- **Local Boundary Interaction:** Rays strictly obey the laws of reflection and refraction at every material interface.
- **Reciprocity:** Light-transport physics is invariant under path reversal, permitting the mathematical inversion of ray paths without loss of validity.

While physical light originates at emitters, the **Synthetic Pinhole Camera** model demands a more computationally tractable approach. Since the overwhelming majority of emitted photons never intersect the camera's Centre of Projection (COP), forward photon tracing is prohibitively expensive for general rendering. We therefore employ **backward ray tracing** (eye-ray tracing): rays are cast from the COP through each pixel of the image plane into the scene. This constitutes a critical *importance-sampling* strategy — computational effort is focused exclusively on light paths that make a measurable contribution to the sensor.

2. The Whitted Recursive Ray Tracing Framework

The Whitted model, introduced by Turner Whitted in 1980, transcended the limitations of local illumination and rasterisation by providing a *recursive* solution to the global lighting equation. By spawning secondary rays at each intersection point, the algorithm naturally captures specular reflection, refraction, and shadow with a single unified architecture.

2.1 Ray Hierarchy

The framework generates a recursive tree of rays for every pixel:

- **Primary Rays:** Emanate from the COP through a specific pixel to identify the first visible surface point P.
- **Shadow Rays:** Directed toward every light source from P to evaluate occlusion.
- **Reflection Rays (R):** Traced in the direction of specular reflection from P.
- **Refracted / Transmitted Rays (T):** Traced through dielectric material boundaries at P.

2.2 The Shading Equation

The total intensity I observed along a ray at surface point P with incident direction **d** is expressed as:

$$I(P, d) = I_{direct} + I_{reflected} + I_{transmitted}$$

To preserve physical plausibility and energy conservation, the recursive components are scaled by material-specific weighting coefficients: $\mathbf{I_{reflected} = k_r \cdot I(Q, R)}$ and $\mathbf{I_{transmitted} = k_t \cdot I(Q, T)}$, where commonly $k_t = 1 - k_r$ to approximate an energy-balanced dielectric.

3. Mathematical Derivation of the Vector Refraction Formula

To implement refractive transparency, we must derive the transmission vector **t** using a vector decomposition approach. The derivation operates in the *plane of incidence* spanned by the incident ray and the surface normal.

Step 1 — Variable Definitions & Snell's Law

- **n**: unit outward surface normal.
- **i**: unit incident ray direction.
- **t**: unit transmission (refraction) direction (to be found).
- η_1, η_2: refractive indices of the incident and transmitting media.
- $\eta = \eta_1 / \eta_2$: the index ratio.

Snell's Law in trigonometric form: $\eta_1 \sin\theta_1 = \eta_2 \sin\theta_2$.

Step 2 — Vector Decomposition of the Incident Ray

We decompose **i** into components parallel and perpendicular to the normal:

- $c_1 = \cos\theta_1 = -\mathbf{n} \cdot \mathbf{i}$
- $\mathbf{i}_{par} = -c_1\mathbf{n}$ (component along normal)
- $\mathbf{i}_{perp} = \mathbf{i} + c_1\mathbf{n}$ (tangential component, magnitude = $\sin\theta_1$)

Step 3 — Plane of Incidence Construction

We define a unit vector **m** in the plane of incidence, perpendicular to **n**: $\mathbf{m} = (\mathbf{i} + c_1\mathbf{n}) / \sin\theta_1$. The transmission vector can then be expressed in the (**m**, **n**) basis as: $\mathbf{t} = \sin\theta_2\, \mathbf{m} - \cos\theta_2\, \mathbf{n}$.

Step 4 — Final Closed-Form Formula

Substituting Snell's Law ($\sin\theta_2 = \eta \sin\theta_1$) and eliminating **m**, the $\sin\theta_1$ terms cancel. Defining $c_2 = \cos\theta_2 = \sqrt{1 - \eta^2(1 - c_1^2)}$, we obtain the **vector refraction formula**:

$$\mathbf{t} = \square\mathbf{i} + (\square c_1 \square c_2)\,\mathbf{n}$$

$$c_2 = \square[1 \square \square^2(1 \square c_1^2)]$$

4. Physical Boundary Conditions & Material Indices

The propagation of the transmitted ray is governed by the refractive index η of the boundary materials, measured relative to vacuum. The table below lists values for common optical media encountered in rendering.

Material	Refractive Index (η)	Typical Application
Air	1.0003	Background medium
Water	1.33	Pool / ocean surfaces
Material	**Refractive Index (η)**	**Typical Application**
Glass	1.45 – 1.65	Windows, lenses, prisms
Salt	1.54	Crystal simulations
Diamond	2.42	Jewellery rendering
Polycarbonate	1.59	VR lens simulation

Total Internal Reflection (TIR)

TIR occurs when a ray travels from a denser medium to a less dense one ($\eta_1 > \eta_2$). As θ_1 exceeds the **critical angle** θ_c, refraction becomes physically impossible. Mathematically, this occurs when the discriminant of c_2 is negative: $\mathbf{1 - \eta^2(1 - c_1^2) < 0}$. In this state c_2 becomes imaginary, transmission is zero, and 100% of incident energy is reflected. In a renderer, the refraction branch is simply skipped and a pure reflection ray is spawned instead.

For the fiber optic values given in the course exercises (n_{core} = 1.50, n_{clad} = 1.48), the critical angle is θ_c = arcsin(1.48/1.50) ≈ **80.6°**. An angle of incidence of 85° inside the fiber exceeds this threshold, confirming that TIR is met and the signal is successfully trapped.

5. Implementation Strategies & Recursive Logic

The architecture of a Whitted-style renderer centres on a **traceRay** function that recursively evaluates the ray hierarchy. Two termination strategies bound the recursion.

5.1 Integrated Shading & Trace Logic (Pseudocode)

```
function shade(mtrl, scene, Q, N, d):
    I = mtrl.ke + mtrl.ka * scene.ambientIntensity
    for each light L in scene:
        atten = L.distanceAttenuation(Q) * L.shadowAttenuation(scene,
        Q) if atten > 0:
            I += atten * (mtrl.diffuseTerm(N,L) + mtrl.specularTerm(N,L,d))
    return I

function traceRay(scene, P, d, depth, weight):
    if depth > MAX_DEPTH or weight <
    MIN_WEIGHT:
        return background_colour
    (t, N, mtrl) = scene.intersect(P,
    d)    if    not    hit:    return
    background_colour Q = P + t * d
    I = shade(mtrl, scene, Q, N,
    d) # --- Reflection ---
    R = reflectDirection(N, -d)
    I += mtrl.kr * traceRay(scene, Q, R, depth+1, weight *
    mtrl.kr) # --- Refraction ---
    if mtrl.kt > 0:
        eta = index_air / mtrl.index if ray_entering else mtrl.index / index_air
        if not isTIR(eta, N, -d):
            T = refractDirection(eta, N, -d)
            I += mtrl.kt * traceRay(scene, Q, T, depth+1, weight * mtrl.kt)
    return I
```

5.2 GLSL Fragment Shader — refractDirection

```
vec3 refractDirection(float eta, vec3 N, vec3 I) {
    // I: incident (pointing toward surface), N: outward
    normal float c1 = dot(-N, I);    // cos(theta_i)
```

```
    float disc = 1.0 - eta*eta*(1.0 - c1*c1);
    if (disc < 0.0) return vec3(0.0);       // TIR — caller uses
    reflection float c2 = sqrt(disc);       // cos(theta_t)
    return eta * I + (eta * c1 - c2) * N;
}
```

5.3 C++ Vector Struct (Minimalist Card-Tracer Style)

```
struct v3 {
    float x, y, z;
    v3 operator+(v3 r) const { return {x+r.x, y+r.y, z+r.z};
    } v3 operator*(float r) const { return {x*r, y*r, z*r}; }
    float operator%(v3 r) const { return x*r.x + y*r.y + z*r.z; } // dot
    v3 operator^(v3 r) const {       // cross
        return {y*r.z-z*r.y, z*r.x-x*r.z, x*r.y-y*r.x};
    }
    v3 operator!() const { return *this * (1.f/sqrtf(*this%*this)); } // normalize
};

v3 refract(v3 I, v3 N, float eta)
    { float c1 = (-N) % I;
    float disc = 1.f - eta*eta*(1.f -
    c1*c1); if (disc < 0.f) return
    v3{0,0,0}; // TIR return I*eta +
    N*(eta*c1 - sqrtf(disc));
}
```

5.4 Recursion Termination

- **Fixed Depth:** Terminate at a constant limit (typically N = 3 to 10 bounces).
- **Adaptive Weight-Based:** Terminate when the cumulative product of k_r/k_t coefficients falls below a threshold (e.g., 0.01), ensuring negligible pixel contributions are discarded early without visual artefacts.

6. Advanced Considerations & Real-World Applications

6.1 Dispersion (Wavelength Dependency)

A common misnomer in introductory texts is the conflation of *dispersion* with *diffraction*. Physically, the variation of the refractive index with wavelength is **dispersion** — the Cauchy or Sellmeier equations model this precisely for optical glass. To simulate chromatic dispersion (e.g., the spectral 'rainbow' in a prism), the renderer must compute three separate indices for the R, G, and B channels, or preferably use full spectral sampling with 400–700 nm wavelength bins. Each wavelength component traces an independent refraction path, producing the characteristic colour fringing.

6.2 Fresnel Equations — Energy Split

In physical reality, every dielectric interface partially reflects and partially transmits light. The exact energy ratio is described by the **Fresnel equations**. The Schlick approximation provides a computationally cheap estimate:

```
float schlick(float cos_theta, float n1, float n2)
    { float r0 = (n1 - n2) / (n1 + n2);
    r0 *= r0;
    return r0 + (1.f - r0) * pow(1.f - cos_theta, 5.f);
}
// Use F as kr; (1-F) as kt for energy conservation.
```

6.3 Evolution Toward Global Illumination

Recursive ray tracing provides the foundational logic for modern **Path Tracing** and **Photon Mapping**. By transitioning from deterministic secondary rays to stochastic importance sampling over the hemisphere, these algorithms resolve complex light paths including colour bleeding, caustics, and subsurface scattering —

phenomena that deterministic Whitted tracing cannot capture.

- **Path Tracing:** Stochastic sampling of the rendering equation; unbiased convergence.
- **Bidirectional Path Tracing (BDPT):** Connects eye and light sub-paths for superior caustics and light-source sampling.
- **Metropolis Light Transport (MLT):** Mutation-based sampling of high-contribution paths.
- **Photon Mapping:** Two-pass algorithm; photon map stores energy at surface interactions.

6.4 Industry Applications

Vector refraction architectures are indispensable across several high-value domains:

- **Automotive Visualisation:** Accurate rendering of headlamp optics, glass roofs, and chrome surfaces in product configurators.
- **Cinematic CGI:** Ocean surfaces, glass objects, and atmospheric effects in feature-film production pipelines (e.g., Arnold, RenderMan).
- **Optical Engineering:** Simulation of camera lens stacks, microscopes, and LiDAR sensor models where sub-millimetre accuracy is mandatory.
- **Scientific Visualisation:** Simulation of Schlieren imaging, X-ray refractive optics, and atmospheric refraction in meteorological models.

7. Historical Development of Ray Tracing

The conceptual roots of ray tracing extend to Arthur Appel's 1968 paper *Some Techniques for Shading Machine Renderings of Solids*, which introduced casting rays from the eye into a scene to determine visibility and compute direct illumination via shadow rays. Appel's model was strictly local — no inter-object reflections were computed — yet it established the fundamental ray-object intersection pipeline that all subsequent work builds upon.

The pivotal advance came in 1980 when Turner Whitted published *An Improved Illumination Model for Shaded Display* in the Communications of the ACM. Whitted extended Appel's single-bounce model into a fully recursive tree, spawning reflection and refraction rays at every dielectric surface. His landmark image of reflective and refractive spheres demonstrated mirror reflections, glass caustics, and crisp shadows that rasterisation-based pipelines could not approach. The image required approximately 74 minutes to render on the hardware of the day.

Throughout the 1980s, the Whitted model was refined and extended. Robert Cook introduced **distributed (stochastic) ray tracing** in 1984, replacing single deterministic secondary rays with distributions of rays to simulate soft shadows, depth of field, motion blur, and glossy reflections. This served as a conceptual bridge from the deterministic Whitted model to the statistical methods that dominate modern rendering.

James Kajiya's 1986 **Rendering Equation** provided the theoretical foundation unifying all light transport algorithms. The equation revealed that Whitted ray tracing solves only a small, analytically tractable subset of this integral — perfect specular reflection and refraction — while ignoring diffuse inter-reflections entirely.

The 1990s and 2000s saw the rise of **photon mapping** (Jensen, 1996) and **bidirectional path tracing** (Lafortune and Willems, 1993). Today, GPU-accelerated path tracing (NVIDIA RTX, 2018) brings interactive ray tracing to consumer hardware, with dedicated RT cores performing bounding-volume hierarchy traversal at hardware speed. The Whitted model remains the pedagogical entry point and conceptual skeleton of every modern renderer.

8. Fresnel Equations — Rigorous Energy Partition

The vector refraction formula determines the *direction* of the transmitted ray, but says nothing about *how much* energy is transmitted versus reflected. This is governed by the **Fresnel equations**, derived from the boundary conditions of Maxwell's equations at a dielectric interface.

8.1 Polarisation Components

Any incident ray can be decomposed into two orthogonal polarisation states: **s-polarisation** (TE — electric field perpendicular to the plane of incidence) and **p-polarisation** (TM — electric field parallel to the plane of incidence). The reflectance for each component is computed from the angle of incidence and transmitted angle. For unpolarised light the net reflectance is the arithmetic mean $\mathbf{R = (R_s + R_p) / 2}$, and transmittance follows from energy conservation: **T = 1 - R**.

At grazing incidence the surface becomes a perfect mirror (R = 1) — this is observable on calm water viewed at a shallow angle, or on glass windows at low sun angles. At normal incidence the reflectance for glass/air is only about 4%, which is why clear glass appears transparent from straight on.

8.2 Schlick Approximation

The full Fresnel computation involves trigonometric functions evaluated per ray. Christophe Schlick (1994) proposed a polynomial approximation accurate to within 1% for most dielectrics, now the de-facto standard in real-time and offline renderer

```
float schlickFresnel(float cosTheta, float n1, float n2) {
    float r0 = (n1 - n2) / (n1 + n2);
    r0 = r0 * r0;     // reflectance at normal
    incidence return r0 + (1.0 - r0) * pow(1.0 -
    cosTheta, 5.0);
}
// In shading loop:
float F = schlickFresnel(dot(N, -I), n_incident,
n_transmitted); float kr = F;// fraction reflected
float kt = 1.0 - F;   // fraction transmitted
```

9. Worked Numerical Example — Ray Through a Glass Cube

To ground the derivation in a concrete calculation, we trace a single ray entering and exiting a glass cube (n = 1.5, surrounded by air n = 1.0) at a 45 degree angle of incidence. This mirrors the classic Track 1 project deliverable.

Entry Interface (Air to Glass)

- Given: angle_i = 45 deg, n1 = 1.0 (air), n2 = 1.5 (glass).
- Snell: sin(angle_t) = (1.0/1.5) x sin(45) = 0.4714 => angle_t = 28.13 deg.
- eta = n1/n2 = 0.6667. c1 = cos(45) = 0.7071, c2 = cos(28.13) = 0.8819.
- Transmission vector: t = 0.6667 i + (0.6667 x 0.7071 - 0.8819) n = 0.6667 i - 0.4110 n.
- Fresnel reflectance (Schlick): R = 0.04 + 0.96 x (1 - 0.7071)^5 = 4.4%.
- Therefore 95.6% of energy is transmitted into the glass.

Exit Interface (Glass to Air)

- At the far face the ray hits at 28.13 deg (same ray, reversed normal).
- Critical angle: angle_c = arcsin(1.0/1.5) = 41.81 deg.
- Since 28.13 < 41.81 deg, TIR does NOT occur — the ray exits successfully.
- Exit angle by Snell (reciprocal): sin(angle_exit) = 1.5 x sin(28.13) = 0.7071 => 45 deg.
- The ray exits parallel to the entry ray with a lateral displacement proportional to slab thickness.
- For a 10 cm glass cube, the lateral displacement at 45 deg incidence is approximately 3.8 cm — a significant error if ignored in a robotic vision system.

10. Comparison of Light Transport Algorithms

The Whitted model is one point in a broad design space of light transport algorithms. The table below summarises the key characteristics of the main approaches encountered in production and research renderers, providing a practical decision matrix for selecting the appropriate strategy.

Algorithm	Year	Caustics	Diffuse GI	Convergence
Whitted RT	1980	Partial	No	Deterministic
Distributed RT	1984	Partial	No	Stochastic
Path Tracing	1986	Poor	Yes	Unbiased
Photon Mapping	1996	Yes	Yes	Biased
BDPT	1993	Good	Yes	Unbiased
MLT	1997	Yes	Yes	Unbiased
RTX / Real-time	2018	Approx.	Approx.	Denoised

11. Dispersion and Chromatic Aberration

In a non-dispersive medium the refractive index is constant across all wavelengths. In reality, all transparent media exhibit **dispersion**: the refractive index varies with wavelength, described by the Cauchy equation n(lambda) = A + B / lambda^2, where A and B are material constants and lambda is the wavelength in micrometres.

For borosilicate glass (BK7), representative values A = 1.5046, B = 0.0042 yield n = 1.5197 at 400 nm (violet) and n = 1.5112 at 700 nm (red). The difference of 0.009 appears small, but across a thick prism it produces the dramatic spectral spread visible in Newton's prism experiments.

11.1 Implementation Strategy

- **RGB approximation:** Use three fixed wavelengths (450 nm blue, 550 nm green, 650 nm red) with separate refractive indices per channel. Simple and fast; captures coarse colour fringing.
- **Spectral sampling:** Sample 16-64 wavelengths between 380-780 nm per ray. Accumulate into a spectral power distribution and convert to XYZ/sRGB at output. Accurate but expensive; standard in scientific and architectural rendering.
- **Hero wavelength sampling:** A compromise — trace one hero wavelength per path with additional wavelengths sampled stochastically. Used in Mitsuba and PBRT v3+.

```
// RGB dispersion via Cauchy equation
float cauchyN(float A, float B, float lambda_um) {
    return A + B / (lambda_um * lambda_um);
}
float n_R = cauchyN(1.5046, 0.0042, 0.650); // red -> 1.5056
float n_G = cauchyN(1.5046, 0.0042, 0.550); // green ->
1.5185 float n_B = cauchyN(1.5046, 0.0042, 0.450); // blue
-> 1.5253

vec3 T_R = refractDirection(n_air/n_R, N,
I); vec3 T_G = refractDirection(n_air/n_G,
N, I); vec3 T_B =
refractDirection(n_air/n_B, N, I);
// Trace three separate rays; combine into RGB output pixel
```

12. Conclusion and Future Directions

This report has presented a rigorous, self-contained treatment of vector-based refraction within the recursive ray tracing paradigm. Starting from the four axioms of geometric optics and backward ray casting, we derived the closed-form transmission vector formula **t = eta*i + (eta*c1 - c2)*n** entirely from Snell's Law via vector decomposition in the plane of incidence. The result is coordinate-free, numerically stable, and directly portable to GPU shader code.

The integration of the refraction formula into the Whitted recursive framework — complete with TIR detection, Schlick-Fresnel energy splitting, and depth/weight-based termination — yields a physically plausible dielectric renderer whose core logic fits in under 30 lines of C++ or GLSL. A worked numerical example quantified the lateral displacement error that would affect a robotic vision system if refraction were ignored.

The historical survey situates the Whitted model within the broader evolution of light transport simulation, from Appel 1968 through to modern GPU-accelerated path tracing. The algorithm comparison table provides a practical decision matrix for selecting an appropriate rendering strategy based on required visual effects and computational budget.

Future Directions

- **Subsurface Scattering (SSS):** Extending the dielectric model to participating media where light scatters inside the volume (skin, marble, milk). Requires volumetric ray marching or diffusion-based approximations.
- **Spectral Rendering:** Full wavelength-resolved simulation for optical engineering applications where colorimetric accuracy is mandatory.
- **Neural Radiance Fields (NeRF):** Implicit scene representations that learn volumetric light transport from images, complementing classical ray tracing for novel-view synthesis.
- **Real-time Denoising:** AI-based denoisers (NVIDIA DLSS, Intel OIDN) allow path tracers to run at 1-4 samples per pixel in real time, enabling physically accurate glass and water in interactive applications.

References

[1] Appel, A. (1968). *Some Techniques for Shading Machine Renderings of Solids.* AFIPS Spring Joint Computer Conference.

[2] Whitted, T. (1980). *An Improved Illumination Model for Shaded Display.* Communications of the ACM, 23(6).

[3] Cook, R.L., Porter, T. & Carpenter, L. (1984). *Distributed Ray Tracing.* ACM SIGGRAPH Computer Graphics, 18(3).

[4] Kajiya, J.T. (1986). *The Rendering Equation.* ACM SIGGRAPH Computer Graphics, 20(4).

[5] Lafortune, E.P. & Willems, Y.D. (1993). *Bi-directional Path Tracing.* Proc. CompuGraphics.

[6] Jensen, H.W. (1996). *Global Illumination using Photon Maps.* Rendering Techniques 96 (EGWR).

[7] Schlick, C. (1994). *An Inexpensive BRDF Model for Physically-Based Rendering.* Computer Graphics Forum, 13(3).

[8] Pharr, M., Jakob, W. & Humphreys, G. (2023). *Physically Based Rendering.* 4th ed. MIT Press.

[9] Shirley, P. & Morley, R.K. (2003). *Realistic Ray Tracing.* 2nd ed. AK Peters.

[10] Glassner, A.S. (1989). *An Introduction to Ray Tracing.* Academic Press.

[11] Scratchapixel 2.0. *Introduction to Ray Tracing: The Refraction Model.* scratchapixel.com.

[12] ENSTA DFI-MI2 Course Notes. *Physics / Computer Vision, Series of Exercises N1.* 2025-2026.